IMPLEMENTATION QUALITY / 27 OCTOBER

Did Develop finish more tasks?

The same model (gpt-6.1-sol) produced a patch with and without Develop. An implementation earned a pass only when it met the task's frozen grading criteria.

Develop completed 0 tasks that Control missed

Control alone passed 0 tasks. Of its 6 grades, 3 passed, 2 failed and 1 remained unresolved. Each task ran once per arm.

Overall implementation grade for each task

	fmt	Svelte	law-config	Personal site	Tokio	Runner-space
Control	Fail	Pass	Pass	Unresolved	Pass	Fail
Develop	Fail	Pass	Pass	Unresolved	Pass	Fail

fmt fixed part of its requirement. Runner-space failed to compile. The site patches matched gold on the primary checks, but their submitted tests timed out.

Completion, elapsed time and tokens by fixture group

NEW TASKS	CONTROL PASSED	DEVELOP PASSED	CHANGE	DEVELOP TIME	DEVELOP TOKENS
Recovered fixtures	2/4 (50%)	2/4 (50%)	0 pp	3.8% slower	23.5% more
Replacement fixtures	1/2 (50%)	1/2 (50%)	0 pp	2.9% slower	25.7% more

pp = percentage points. Time and total tokens compare Develop with Control. Tokens include cached input. Billed dollars were unavailable.

What the reviewers could distinguish

Correctness tied on 6/6 tasks. Maintainability tied on 5/6. The site reviewer preferred Control's tests for navigation, cleanup and keyboard ownership. The submitted site tests timed out for both arms.

Earlier full-workflow evidence

Both arms passed all 5 completed historical pairs. Only 2/8 selected cases completed that workflow. Reviewers modestly preferred Develop's Flask and Hugo tests. The six new pairs measured edits made without repository tools or test feedback.

TASK COMPLETION / 27 OCTOBER

How far did each patch get?

Read across from the starting code (base) to the two patches and the merged human fix (gold). Scores use each task's own tests. A build failure leaves behavior unknown.

Recovered fixtures

TASK	BASE	CONTROL	DEVELOP	GOLD
fmt	5/13	7/13	7/13	13/13
Svelte	0/2 derived	2/2 derived	2/2 derived	2/2 derived
law-config	2/7	7/7	7/7	7/7
Personal site	0/5 browsers	3/5 browsers	3/5 browsers	3/5 browsers

fmt / Null-pointer formatting

Both fixed direct formatting and conversion to a string, recovering 2 of the 8 checks the base missed. The remaining six paths still crashed, including surrounding text, bounded output and size calculation.

Svelte / Visibility of speculative state

Both fixed the same fork-visibility regression in two rendering modes. All eight independent checks already passed on base. Only the two gold-derived checks distinguished the fix.

law-config / SQLite lock retries

Both recovered all five missing scenarios, including the default wait, retry limits and recovery when a writer released its lock. Neither submitted README test program was executed.

Personal site / Reconnect arcade controls

Both restored navigation in Chromium, Firefox and Mobile Chrome. Both reproduced gold's failures in WebKit and Mobile Safari. Their submitted tests timed out at 420 seconds, leaving both overall grades unresolved.

Replacement fixtures

TASK	BASE	CONTROL	DEVELOP	GOLD
Tokio	API missing	4/4 runtime	4/4 runtime	4/4 runtime
Runner-space	4/11	Build failed	Build failed	11/11

Tokio / Mutable access to a stored value

Base lacked the method, so the runtime checks could not start. Both added exclusive mutable access, passed all four new runtime checks and 26 existing tests, and correctly rejected a shared borrow.

Runner-space / Safe filesystem removal

Base passed 4 of 11 removal checks and gold passed all 11. Both patches failed to compile. Whether either preserved a replacement target or detected incomplete removal remained untested.

Source / candidate test observations

Source / base and gold checks

CODE AND TESTS / 27 OCTOBER

Where did patch quality differ?

The graders saw anonymous submissions. A stronger regression test could earn a preference even when both patches delivered the same measured behavior.

Recovered fixtures

TASK	IMPLEMENTATION AND TEST QUALITY	DEVELOP TIME	DEVELOP TOKENS
fmt	Both routed null through the same pointer writer. Develop tested null C strings and mutable strings. Control tested raw-pointer output and more string conversions. Both patches still crashed on the same six paths.	2.2% faster	24.7% more
Svelte	Develop tested whether an unrelated update could expose pending state to the same observer. Control checked ordinary and eager reads together and verified that an unrelated flush sent no notification. Both submitted suites passed.	35.9% slower	23.0% more
law-config	Control covered contention at commit and kept post-commit optimization errors separate. Develop tested a wider set of lock diagnostics and synchronized lock release more precisely. These are differences in the submitted code.	17.0% faster	21.5% more
Personal site	Control added tests for child-initiated navigation, unmounting an inaccessible frame and restored keyboard ownership. Develop separated host and frame subscriptions and guarded stale focus callbacks. Control won the maintenance judgment on source inspection.	12.4% slower	25.2% more

Replacement fixtures

TASK	IMPLEMENTATION AND TEST QUALITY	DEVELOP TIME	DEVELOP TOKENS
Tokio	Both implemented the same accessor and documented its safety. Control tested repeated mutation. Develop tested extracting from an empty cell. Both submitted suites passed.	2.2% faster	24.6% more
Runner-space	Control used operation-local dependencies. Develop used shared mutable test hooks. The blind reviewers tied. Later compiler logs identified four additional type errors in Develop's tests, on top of the unused import that blocked both builds.	5.2% slower	26.8% more

NEXT EXPERIMENTS

What should the next trial test?

Use fresh matched pairs, preserve failures, and change one element at a time. The observed differences specify what to measure in a follow-up.

1. Would compiler feedback help patches reach runtime tests?

Both runner-space builds stopped on the unused inspectTarget import (TS6133). Develop's test code also had 4 implicit-any errors (TS7006). Neither arm reached a runtime suite.

On fresh TypeScript tasks, give both arms one compiler report and equal repair time. Compare with the existing tool-free procedure. Count builds and runtime passes separately, then compare time and tokens.

2. Which review prompt catches incomplete behavior?

Both fmt patches applied. Control passed 7/13 and Develop passed 7/13. Both still failed on bounded output and surrounding literal text.

On fresh C++ tasks, compare current Develop with a short review prompt to trace the change through each public entry point. Freeze independent checks for boundary behavior before the arms run.

3. Can a shorter review preserve quality?

The four review calls per new task accounted for the following context.

FIXTURES	PAIRS	CONTROL REVIEW TOKENS	DEVELOP REVIEW TOKENS
Recovered fixtures	4	598,375	742,094
Replacement fixtures	2	286,337	360,155

Test shorter Develop review inputs at the same review depth. Compare phase time and usage. Require unchanged acceptance and blind-review quality, then compare resource use.

RECOVERED FIXTURES / 27 OCTOBER

Time and tokens in the new trials

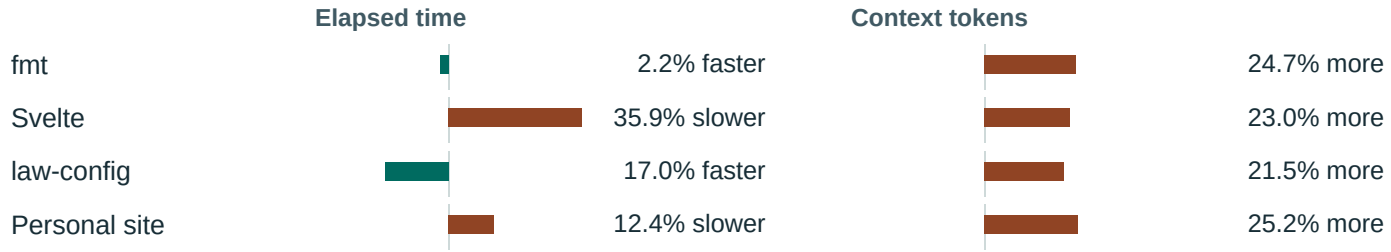
Recovered fixtures · 4 paired tasks. Each row counts the planning, implementation, review and final response for one arm. The tables include unsuccessful submissions. Aggregate percentages compare sums across tasks.

PROJECT	ARM	ELAPSED	UNCACHED IN	CACHED IN	OUTPUT	CONTEXT
fmt	Control	90.3 s	234,054	19,456	4,695	258,205
	Develop	88.3 s	299,984	16,256	5,706	321,946
Svelte	Control	140.3 s	226,035	37,760	6,444	270,239
	Develop	190.7 s	309,815	15,104	7,600	332,519
law-config	Control	258.4 s	226,435	45,312	14,503	286,250
	Develop	214.4 s	280,660	52,864	14,328	347,852
Personal site	Control	166.1 s	199,160	45,312	11,749	256,221
	Develop	186.6 s	254,902	52,864	12,951	320,717
4-pair sum	Control	655.1 s	885,684	147,840	37,391	1,070,915
	Develop	680.0 s	1,145,361	137,088	40,585	1,323,034

Develop minus control across these tasks was +259,677 uncached input, -10,752 cached input, +3,194 output tokens.

Context = uncached input + cached input + output, including reasoning. These counts show model workload. A dollar figure would require billable usage and applicable rates.

Time and context relative to Control



RECOVERED FIXTURES / 27 OCTOBER

Which phases used the most context?

Recovered fixtures · 4 paired tasks. Every call received the source material and instructions for its arm. The figures below count uncached input, cached input and output summed across tasks. Concurrent calls each contribute usage.

PHASE	CONTROL	DEVELOP	CHANGE IN TOKENS
Planning	143,805	179,021	+35,216
Plan review · intent	144,145	179,404	+35,259
Plan review · artifact	144,296	179,696	+35,400
Implementation	161,408	196,844	+35,436
Final review · intent	154,856	191,692	+36,836
Final review · artifact	155,078	191,302	+36,224
Final response	167,327	205,075	+37,748

The ledger contains 56 of 56 expected phase receipts. 56 acknowledge the instructions. Recorded tool items total 0.

External and user-owned tasks

STRATUM	PROJECTS	PASS · C / D	ELAPSED TIME	CONTEXT TOKENS
External	fmt, Svelte	1/2 · 1/2	21.0% slower	23.8% more
User-owned	law-config, Personal site	1/2 (1 unresolved) · 1/2 (1 unresolved)	5.5% faster	23.2% more

Each pass count uses all released tasks in that group. User-owned projects carry additional prior-context risk. These small groups describe the observed tasks.

RECOVERED FIXTURES / 27 OCTOBER

What each arm produced

fmt / fmtlib/fmt/4993

Merged human reference / fmtlib/fmt #4993

External project · possible prior exposure

cpp-v1 · 89,873 packet bytes · 20 min per arm · Develop then Control

Formatting nullptr through compiled format strings and to_string should produce the same pointer representation as ordinary runtime formatting, rather than treating it as a null C string or crashing. Preserve formatting with surrounding literal text and through format_to, format_to_n and formatted_size, while retaining existing behavior for actual strings and pointers.

BASE REFERENCE	GOLD REFERENCE	BLIND CORRECTNESS	BLIND MAINTAINABILITY
5/13	13/13	Tie	Tie

Candidate results

ARM	OUTCOME	TEST EVIDENCE OR APPLICATION RESULT
Control	Fail · 7/13	13 held-out checks. Pass for format-test. Failed or incomplete for compile-test.
Develop	Fail · 7/13	13 held-out checks. Pass for format-test. Failed or incomplete for compile-test.

What the evidence covers

Both gained two checks over base. Composite formatting, all three bounded-output cases, format_to and formatted_size still failed in both.

Both native builds succeeded. After blind grading, the CTest logs confirmed that compile-test segfaulted on both patches and format-test passed. The six failing held-out processes exited 139.

RECOVERED FIXTURES / 27 OCTOBER

What each arm produced

Svelte / sveltejs/svelte/19076

Merged human reference / sveltejs/svelte #19076

External project · possible prior exposure

js-v2 · 89,970 packet bytes · 75 min per arm · Develop then Control

Speculative forks must not expose uncommitted state through $inspect or $state.eager. Starting a fork that changes state must leave observations in the committed application unchanged while that fork is pending. Discarding the fork must not publish its values. Committing it must make the committed values observable through the normal update cycle. Preserve ordinary updates outside forks. This task concerns observable fork visibility, without prescribing a scheduling or dependency-tracking implementation.

BASE REFERENCE	GOLD REFERENCE	BLIND CORRECTNESS	BLIND MAINTAINABILITY
8/8 independent; 0/2 gold-derived	8/8 independent; 2/2 gold-derived	Tie	Tie

Candidate results

ARM	OUTCOME	TEST EVIDENCE OR APPLICATION RESULT
Control	Pass · 2/2 gold-derived	8/8 independent, 2/2 gold-derived, 5/5 native.
Develop	Pass · 2/2 gold-derived	8/8 independent, 2/2 gold-derived, 5/5 native.

What the evidence covers

Both patches isolate speculative state and pass all observed checks. Each fixes the two reference-derived regressions. Neither blind grader found a material quality difference.

The scorer ran 8 independent, 2 gold-derived and 5 fixed native assertions. Both builds and both separately run authored regression suites passed.

RECOVERED FIXTURES / 27 OCTOBER

What each arm produced

law-config / justinthelaw/law-config/20

Merged human reference / justinthelaw/law-config #20

User-owned project · prior exposure unknown

shell-v2 · 86,164 packet bytes · 75 min per arm · Develop then Control

Make orphan-pruning database writes tolerate temporary SQLite writer contention. Keep the existing pruning behavior and default per-attempt busy wait, add documented configuration of the busy timeout and finite retry count, and retry lock-related write failures only within those configured bounds. If a competing writer releases its lock before exhaustion, pruning should complete. Exhaustion must report failure; non-lock SQLite errors must fail without repeated lock retries. Preserve retained records and transactional consistency. Evaluation must use only disposable synthetic SQLite data and fixture paths; never real account, session, rollout or history data.

BASE REFERENCE	GOLD REFERENCE	BLIND CORRECTNESS	BLIND MAINTAINABILITY
2/7	7/7	Tie	Tie

Candidate results

ARM	OUTCOME	TEST EVIDENCE OR APPLICATION RESULT
Control	Pass · 7/7	Held-out synthetic SQLite scenarios. Syntax checks are reported separately.
Develop	Pass · 7/7	Held-out synthetic SQLite scenarios. Syntax checks are reported separately.

What the evidence covers

Both patches passed seven synthetic SQLite cases and both syntax checks. The blind reviewers found complementary diagnostic and regression coverage, with no overall preference.

Both submitted large regression programs inside the README. The grader ran its own frozen scenarios. It did not execute the two README programs. One documented interface needed a citation-backed manual mapping before its first behavior run.

RECOVERED FIXTURES / 27 OCTOBER

What each arm produced

Personal site / justinthelaw/justinthelaw/413

Merged human reference / justinthelaw/justinthelaw #413

User-owned project · prior exposure unknown

js-v2 · 89,976 packet bytes · 75 min per arm · Control then Develop

Make arcade input controls robust across iframe navigation and reopening. Cleanup must not leave an uncaught SecurityError when an admitted iframe navigates cross-origin; controls must reconnect when a same-origin document returns. Preserve same-origin admission, typing protections, focus/input ownership, synchronous cancellation of diagonal input, existing exports and React state behavior. Host and iframe touch detection must remain correct across player reopening. Improve separation of input-bridge lifecycle responsibilities without changing key-delivery behavior.

BASE REFERENCE	GOLD REFERENCE	BLIND CORRECTNESS	BLIND MAINTAINABILITY
0/5 browsers	3/5 browsers	Tie	Control

Candidate results

ARM	OUTCOME	TEST EVIDENCE OR APPLICATION RESULT
Control	Unresolved · 3/5 browsers	3/5 browser projects, 13/15 strict browser and native checks. Formal comparison with gold unresolved. Overall implementation score unverified.
Develop	Unresolved · 3/5 browsers	3/5 browser projects, 13/15 strict browser and native checks. Formal comparison with gold unresolved. Overall implementation score unverified.

What the evidence covers

Both patches built and passed all ten native controls and three of five held-out browsers. WebKit and Mobile Safari reproduced the gold cross-origin failures. Control added more targeted tests of child-initiated navigation, cleanup and restored input ownership.

Both matched gold on all 15 primary outcomes in a post-blind signature comparison. The submitted tests hit the 420-second limit, leaving overall grades and formal gold comparators unresolved. Correctness tied. The maintainability reviewer preferred Control.

RECOVERED FIXTURES / 27 OCTOBER

What did the blind reviewers find?

Recovered fixtures · 4 paired tasks. Two reviewers separately assessed correctness and maintainability. The labels A and B were assigned anonymously. The key below identifies the arms for each judgment.

fmt

Correctness / Tie

A = control · B = develop

Identical observed behavior, with 7 of 13 checks passing for each. Neither fulfills the complete requirement.

Maintainability and tests / Tie

A = control · B = develop

Both submissions have identical observed results: 7 of 13 behaviors pass, and neither satisfies the full requirement. No material overall advantage is established.

Both add meaningful truncation, zero-capacity, output-position, and sentinel boundaries. B additionally distinguishes nullptr from an actual null C string and tests mutable strings and nullptr lvalues. A additionally tests raw-pointer format_to and broader to_string preservation. These are complementary strengths, not evidence that either implementation succeeds.

Svelte

Correctness / Tie

A = control · B = develop

Both implementations apply and pass identical supplied behavioral suites. No material behavioral advantage is demonstrated.

Maintainability and tests / Tie

A = control · B = develop

Identical passing behavioral evidence. Test and maintenance differences do not establish a clear overall winner.

Both meaningfully test conditional dependency preservation across discard and switching after commit. B additionally mutates the directly observed source inside the fork and forces the same inspector to rerun through unrelated committed updates—a stronger visibility boundary. A separately checks ordinary and eager output plus an unrelated flush without observer notifications.

RECOVERED FIXTURES / 27 OCTOBER

What did the blind reviewers find?

Recovered fixtures · 4 paired tasks. Two reviewers separately assessed correctness and maintainability. The labels A and B were assigned anonymously. The key below identifies the arms for each judgment.

law-config

Correctness / Tie

A = control · B = develop

Both submissions pass the same seven offline behavioral checks and supplied syntax checks. Neither has demonstrated overall behavioral superiority.

Maintainability and tests / Tie

A = control · B = develop

Both applicable implementations passed the same seven offline behavioral checks and syntax checks. The supplied evidence does not establish an overall winner.

Both add meaningful synthetic contention, exhaustion, retention, configuration, and rollback checks. A additionally targets COMMIT-time contention, exact lock-worded trigger errors, and post-commit optimization separation. B provides deterministic release gating and extensive diagnostic-classification boundaries. A's recovery synchronization is less deterministic; B lacks explicit COMMIT-contention and optimization-boundary tests.

Personal site

Correctness / Tie

A = develop · B = control

Observed behavior and build results are identical. Source-level and regression-test differences do not establish a material correctness advantage.

Maintainability and tests / Control

A = develop · B = control

Prefer B for materially more targeted regression tests, not superior observed behavior.

B meaningfully tests child-initiated navigation without src-observer assistance, inaccessible-frame unmount cleanup, and restored typing/focus ownership. Both cover diagonal cancellation and host/iframe touch after reopening. A additionally tests diagonal cancellation after reload and repeated touch respecting hidden controls.

REPLACEMENT FIXTURES / 27 OCTOBER

Time and tokens in the new trials

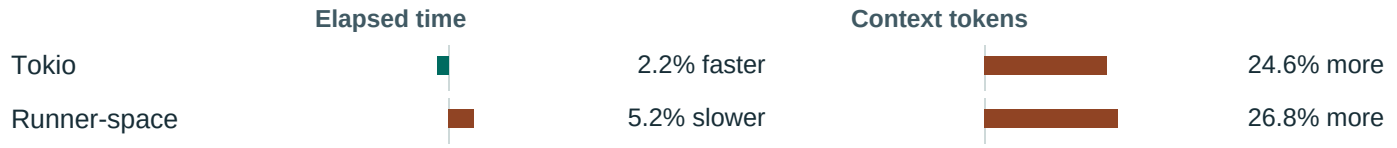
Replacement fixtures · 2 paired tasks. Each row counts the planning, implementation, review and final response for one arm. The tables include unsuccessful submissions. Aggregate percentages compare sums across tasks.

PROJECT	ARM	ELAPSED	UNCACHED IN	CACHED IN	OUTPUT	CONTEXT
Tokio	Control	95.1 s	203,826	45,312	3,963	253,101
	Develop	93.0 s	265,662	45,312	4,486	315,460
Runner-space	Control	219.3 s	199,854	45,312	12,871	258,037
	Develop	230.7 s	257,699	52,864	16,509	327,072
2-pair sum	Control	314.4 s	403,680	90,624	16,834	511,138
	Develop	323.7 s	523,361	98,176	20,995	642,532

Develop minus control across these tasks was +119,681 uncached input, +7,552 cached input, +4,161 output tokens.

Context = uncached input + cached input + output, including reasoning. These counts show model workload. A dollar figure would require billable usage and applicable rates.

Time and context relative to Control



REPLACEMENT FIXTURES / 27 OCTOBER

Which phases used the most context?

Replacement fixtures · 2 paired tasks. Every call received the source material and instructions for its arm. The figures below count uncached input, cached input and output summed across tasks. Concurrent calls each contribute usage.

PHASE	CONTROL	DEVELOP	CHANGE IN TOKENS
Planning	68,814	86,873	+18,059
Plan review · intent	69,070	86,937	+17,867
Plan review · artifact	69,223	87,240	+18,017
Implementation	75,635	94,822	+19,187
Final review · intent	73,930	92,941	+19,011
Final review · artifact	74,114	93,037	+18,923
Final response	80,352	100,682	+20,330

The ledger contains 28 of 28 expected phase receipts. 28 acknowledge the instructions. Recorded tool items total 0.

External and user-owned tasks

STRATUM	PROJECTS	PASS · C / D	ELAPSED TIME	CONTEXT TOKENS
External	Tokio	1/1 · 1/1	2.2% faster	24.6% more
User-owned	Runner-space	0/1 · 0/1	5.2% slower	26.8% more

Each pass count uses all released tasks in that group. User-owned projects carry additional prior-context risk. These small groups describe the observed tasks.

REPLACEMENT FIXTURES / 27 OCTOBER

What each arm produced

Tokio / tokio-rs/tokio/8686

Merged human reference / tokio-rs/tokio #8686

External project · possible prior exposure

rust-v2 · 89,904 packet bytes · 20 min per arm · Control then Develop

Add mutable access to a SetOnce value without consuming the cell. Provide SetOnce::get_mut for an exclusive mutable borrow of the cell, analogous to the existing OnceCell accessor. An uninitialized cell returns None. An initialized cell returns a mutable reference to the stored value, and mutations remain visible through subsequent existing accessors. Preserve initialization, ownership and destruction behavior. No change to AsyncFd is requested.

BASE REFERENCE	GOLD REFERENCE	BLIND CORRECTNESS	BLIND MAINTAINABILITY
Missing API at compile	4/4 runtime, exclusive borrow verified	Tie	Tie

Candidate results

ARM	OUTCOME	TEST EVIDENCE OR APPLICATION RESULT
Control	Pass · 4/4 runtime	Baseline pass, behavior pass, exclusive borrow verified.
Develop	Pass · 4/4 runtime	Baseline pass, behavior pass, exclusive borrow verified.

What the evidence covers

Both added the same exclusive accessor. Both preserved the stored value and its mutations across initialization, access and ownership transfer. Both blind reviewers recorded a tie.

Both passed all four replacement runtime checks and all 26 protected native tests. Both rejected the shared-owner compile probe with E0596 as expected, and both submitted regression suites exited successfully.

REPLACEMENT FIXTURES / 27 OCTOBER

What each arm produced

Runner-space / justinthelaw/maximize-github-runner-space/50

Merged human reference / justinthelaw/maximize-github-runner-space #50

User-owned project · prior exposure unknown

js-v2 · 89,816 packet bytes · 75 min per arm · Control then Develop

Make filesystem-removal failure handling detect a target that has changed since safety validation. If a path is observed to refer to a different target or link kind before removal or a privileged retry, stop that operation safely instead of removing the replacement. Report failure when removal returns but the target remains. Preserve ordered results, best-effort processing of other targets, failed-operation output and existing public inputs and outputs. The concurrency scope is changes observable at the existing platform-adapter boundaries, not an atomic guarantee against arbitrary external mutations between operating-system calls. Test only disposable fixtures and mocked privileged operations.

BASE REFERENCE	GOLD REFERENCE	BLIND CORRECTNESS	BLIND MAINTAINABILITY
4/11	11/11	Tie	Tie

Candidate results

ARM	OUTCOME	TEST EVIDENCE OR APPLICATION RESULT
Control	Fail · Build failed	TypeScript compilation exited 2. No behavior, output or native tests executed.
Develop	Fail · Build failed	TypeScript compilation exited 2. No behavior, output or native tests executed.

What the evidence covers

Both patches applied and both failed the TypeScript build on an unused inspectTarget import. The replacement behavior, output and native suites never ran. Neither blind reviewer preferred a patch.

After blind review, compiler logs confirmed one shared production error. Develop also introduced four implicit-any errors in its test code. Neither authored suite reached execution.

REPLACEMENT FIXTURES / 27 OCTOBER

What did the blind reviewers find?

Replacement fixtures · 2 paired tasks. Two reviewers separately assessed correctness and maintainability. The labels A and B were assigned anonymously. The key below identifies the arms for each judgment.

Tokio

Correctness / Tie

A = control · B = develop

Both provide identical accessor logic and pass the supplied behavioral and baseline checks. No material correctness advantage is evidenced.

Maintainability and tests / Tie

A = control · B = develop

The implementations are functionally identical, and the supplied scoring evidence shows no material correctness or maintenance advantage.

Both supplemental suites exited successfully and cover meaningful boundaries: empty access, mutation visibility through get/wait/into_inner, rejected reinitialization, and exactly-once destruction or ownership transfer. A additionally exercises repeated mutable access; B explicitly checks empty into_inner. Their constructor/drop-path combinations differ but establish no material overall advantage.

Runner-space

Correctness / Tie

A = develop · B = control

Neither submission demonstrates complete fulfillment. Both builds exited 2 with authored-test compilation failures. No candidate suites executed. This is not evidence of a runtime behavioral failure.

Maintainability and tests / Tie

A = develop · B = control

Both builds exited 2 with authored-test compile failures. No suites executed. Neither has demonstrated behavioral superiority.

Both add meaningful disposable-fixture tests for replacement preservation, no-op removal, retry outcomes, and ordered best-effort processing. A uniquely tests separate-helper identity retention and link-to-directory replacement; B tests the Windows uninstaller boundary. Neither added suite compiled successfully.

Compiler logs checked after blind review / Both runner-space builds stopped on the unused inspectTarget import (TS6133). Develop's test code also had 4 implicit-any errors (TS7006). Neither arm reached a runtime suite.

INSTRUCTION-ASSISTED / 27 OCTOBER

How the new trials ran

Each arm received excerpts from the same frozen base and returned exact text edits. The model could read its packet and peer feedback. It had no repository tools or test results.

Treatment and control

Both used gpt-6.1-sol at medium effort and the same task requirement, base packet, output schema, time limit and review roles. Develop also received the selected frozen Develop instructions, 48,714 bytes on each of seven calls. Each phase had to return the supplied instruction receipt.

Seven model calls in each arm

A fresh thread produced a plan. Two planning reviewers independently checked intent and the proposed implementation. A fourth thread proposed edits. Two evidence reviewers inspected the proposal. A final thread sealed the returned edits. The two reviewers ran concurrently at each checkpoint.

What counted as a pass

Anonymous artifacts went to separate evaluators. They first applied the submitted edits to the untouched base. An empty or non-applying artifact earned no implementation credit and received no candidate test score. Applied artifacts faced the case's admitted checks. A separate blind judgment compared correctness and maintainability. Cases use different suites and thresholds.

Timing, scope and reference limits

Each task ran once per arm. Elapsed time captures the model and review workflow. The resource table uses the reported arm usage. Setup, evaluator runs and independent judging are outside those totals. The human reference contributes behavior and code, with unknown implementation time and usage.

Gold is the original merged change. Prior exposure varies by project. Small, purposefully selected samples and one pair per task limit conclusions about other work. The recovered and replacement fixtures use different acceptance evidence. Each has its own score and resource summary. Historical and source-packet execution also differ in tool access, context and review procedure.

COVERAGE / 27 OCTOBER

Which work remains open?

The original eight-case slate and its retry allowances belong to the historical full workflow. The new source-packet runs measure a separate task with different tools and instructions.

Full native workflow

Zero new full-workflow scored pairs. The pinned Linux client had no supported device sign-in. The signed-in alternate passed 224 executor checks but its native process launcher was denied a nested namespace in all four role profiles.

Selected tasks missing a full native pair

PROJECT	26 OCT NATIVE STATE	NEW PACKET FIXTURES	CONTROL / DEVELOP
Tokio	Prepared	Replacement fixtures	Pass · Pass
fmt	Interrupted	Recovered fixtures	Fail · Fail
Svelte	Prepared	Recovered fixtures	Pass · Pass
Personal site	Not admitted	Recovered fixtures	Unresolved · Unresolved
law-config	Interrupted	Recovered fixtures	Pass · Pass
Runner-space	Limited preparation	Replacement fixtures	Fail · Fail

Requests and Cobra completed historical native pairs. The table tracks the six other selected tasks. A new packet result leaves its native experiment open.

HISTORICAL WORKFLOW / 26 OCTOBER

Both arms passed all five tasks

Develop finished two tasks faster and three slower. It used more context on every task. Neither arm showed better production behavior on the tested paths.

Develop compared with control

TASK / SLATE	BOTH PASSED	BLIND ASSESSMENT	ELAPSED TIME	CONTEXT TOKENS
Requests Selected	7/7	Mixed maintenance and test tradeoffs	**31.6% slower** +97.5 s	**55.6% more** +369,696
Cobra Selected	13 leaf subcases	Mixed overall with complementary tests	**27.3% slower** +70.1 s	**55.8% more** +434,485
Starlette Supplemental	12/12	Complementary boundary tests	**1.8% faster** -4.5 s	**52.9% more** +220,199
Flask Supplemental	17/17	Develop's tests modestly preferred	**12.8% faster** -42.4 s	**57.9% more** +346,002
Hugo Supplemental	6 API + 1 template	Develop's tests modestly preferred	**8.1% slower** +59.7 s	**45.4% more** +891,589

Two selected pairs versus all five completed pairs

COMPLETED PAIRS	BOTH PASSED	ELAPSED TIME	CONTEXT TOKENS
Selected (2 pairs)	2 of 2 tasks	**29.6% slower**	**55.7% more**
All historical (5 pairs)	5 of 5 tasks	**9.6% slower**	**51.2% more**

Percentages use control as the baseline. The two summaries compare sums across tasks. Time includes reviews and waiting. The historical resource table gives the measurements for each arm.

Billed dollars are unavailable. Context counts uncached input, cached input and output.

Roux chose the design and interpreted the results. The target fell from 20 cases to eight after some outcomes were known. Only two selected cases finished. Each of the five Python or Go tasks had one pair, limiting conclusions about other tasks.

HISTORICAL WORKFLOW / 26 OCTOBER

One task, two isolated arms

Compare Codex with and without Develop under the same review process. The merged human PR supplies a third reference for behavior and patch quality.

CONTROL

Clean base and repository guidance. Develop is inaccessible.

DEVELOP

Same clean base and guidance, plus the frozen Develop package.

EVALUATOR

Separate gold checkout, hidden acceptance tests and arm mapping.

1. Freeze and admit each case

Verify the gold merge and its first parent, requirement and toolchain hashes. Run base and gold checks offline. Keep the worker prompt free of PR identifiers and solution hints.

2. Match execution and test the boundaries

Use CLI 0.159.2, gpt-6.1-sol at medium effort and Develop 0.9.0. Run arms serially with equal resources and time limits. Each gets one synthetic base commit. Deny command egress, sibling, evaluator and auth paths for workers, reviewers and descendants.

3. Measure the full workflow

Each arm in the five completed pairs used one implementation root, two planning reviewers and two final reviewers. Start the clock before instruction loading. Reconcile final cumulative usage once per native thread. Retain waits, retries, partials and missing values.

4. Grade anonymous patches

Check acceptance and new regressions. Hide arm identities and timing from graders. Compare behavior, tests, security and maintenance against gold. Use targeted reproductions for disagreements and retain uncertainty when evidence is inconclusive.

Limits include possible style leakage or memorization, prior gold exposure for acceptance authors, and retrospective tool-event auditing.

HISTORICAL WORKFLOW / 26 OCTOBER

Time and tokens by task

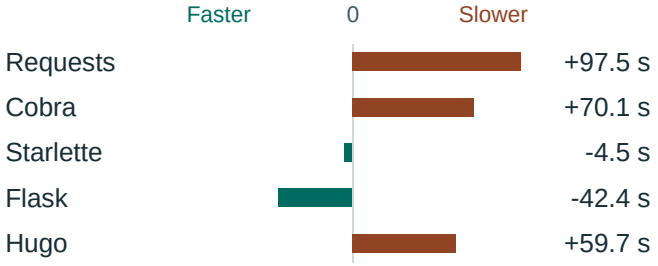
Each row includes the implementation and review work for one arm. Seconds run from release through final receipt, including waits. Token counts come from native usage.

Five completed historical pairs

TASK	ARM	SECONDS	UNCACHED IN	CACHED IN	OUTPUT	CONTEXT
Requests	Control	308.8	114,318	539,648	10,372	664,338
	Develop	406.2	176,924	840,448	16,662	1,034,034
Cobra	Control	256.7	132,786	638,208	7,700	778,694
	Develop	326.8	172,065	1,029,632	11,482	1,213,179
Starlette	Control	247.9	112,357	297,088	6,475	415,920
	Develop	243.4	171,966	454,912	9,241	636,119
Flask	Control	331.9	133,423	456,576	7,574	597,573
	Develop	289.5	154,205	777,344	12,026	943,575
Hugo	Control	734.3	174,654	1,776,768	13,385	1,964,807
	Develop	794.0	212,316	2,627,200	16,880	2,856,396
Five-pair sum	Control	1,879.6	667,538	3,708,288	45,506	4,421,332
	Develop	2,059.9	887,476	5,729,536	66,291	6,683,303

Elapsed time change

Develop minus control, in seconds



Additional context by type

Share of the 2,261,971 extra tokens



Uncached input	+219,938 (9.7%)
Cached input	+2,021,248 (89.4%)
Output	+20,785 (0.9%)

Billed dollars are unavailable. Cached and uncached input are counted separately. Output includes reasoning. Coordinator, grader and human reference usage are unknown and excluded.

HISTORICAL WORKFLOW / 26 OCTOBER

How did the patches compare?

Acceptance is measured per case. Qualitative preferences below come from the historical blind assessments. Different test sets make pooled test counts misleading.

Requests

SELECTED / BOTH PASSED 7/7

Both handled file-like proxies and passed 243 socket-free native checks, with 13 skipped. The assessors found mixed maintenance and test tradeoffs. Full HTTP coverage was incomplete.

Cobra

SELECTED / BOTH PASSED 13 LEAF SUBCASES

Both satisfied the validator requirement and passed the full Go suite. Gold retained more contextual diagnostics. The two agents added complementary regression tests.

Starlette

SUPPLEMENTAL / BOTH PASSED 12/12

Both agents preserved a repeated-slash root-path boundary that gold broke. Their production fixes matched. The improvement over gold was reproduced after the trials.

Flask

SUPPLEMENTAL / BOTH PASSED 17/17

Both preserved explicit host and port overrides with an invalid configured port. Gold raised an error in this post-hoc check. Assessors modestly preferred Develop's regression tests.

Hugo

SUPPLEMENTAL / BOTH PASSED 6 API + 1 TEMPLATE

Runtime behavior matched across both arms. Each improved template regression coverage over gold. Assessors favored Develop's additional pipeline and conversion boundary tests.

Only the tested paths support these comparisons.

HISTORICAL WORKFLOW / 26 OCTOBER

Where did the eight cases stand?

These are the dispositions on 26 October, before the source-packet trials. The first five projects are external. The last three are user-owned and have prior-context risk. Case labels link to the gold PRs.

CASE / GOLD PR	FIRST / LIMIT	LAST VERIFIED STATE
Requests #7505 Complete	Control 40 min / arm	Both arms passed all seven acceptance checks.
Cobra #2397 Complete	Develop 20 min / arm	Both arms passed 13 leaf subcases and the full Go suite.
Tokio #8686 Prepared	Control 20 min / arm	No arms. Worker baseline 26/26. The new API was absent on base.
fmt #4993 Interrupted	Develop 20 min / arm	Develop partial passed 13/13. Final review hit a service limit. Control unstarted.
Svelte #19076 Prepared	Develop 75 min / arm	No arms. Independent checks passed 8/8 on both references. Gold-derived checks went from 0/2 to 2/2.
Personal site #413 Not admitted	Control 75 min / arm	Offline install and build passed. Five browser projects still needed runtime admission.
law-config #20 Interrupted	Develop 75 min / arm	Develop stopped before edits. Control unstarted. Acceptance was 2/7 on base and 7/7 on gold.
Runner-space #50 Limited preparation	Control 75 min / arm	No arms. Behavioral checks went from 3/9 to 9/9. Six native permission failures affected both references.

Six selected pairs remained. fmt and law-config each retained one fresh pair retry. Outside the selected slate, Click's pilot was ambiguous, Vue exhausted its retry and FastAPI lacked a completed control arm.

HISTORICAL WORKFLOW / 26 OCTOBER

What did the first five pairs suggest?

The five completed tasks each ran once per arm. Test one change at a time in fresh matched pairs, with the same review roles, time limits and blind grading in both arms.

01 Could smaller review packets save time and context?

On Requests and Cobra, Develop took 167.5 s longer in total and used 804,181 more tokens. Both tasks had mixed blind assessments.

Measure time and tokens by phase. Test a shorter packet against current Develop. Keep it if time and tokens fall, acceptance still passes and graders find no quality loss or new regressions.

02 Which test guidance earns its cost?

Assessors favored Develop's tests on Flask and Hugo. Develop finished Flask 42.4 s faster and Hugo 59.7 s slower. Develop used more context on both.

Compare focused boundary-test guidance with current instructions. Check whether new tests catch the override or pipeline failure. Blind-grade coverage and record elapsed time and tokens.

03 Can the fixture distinguish a working change?

On Svelte, independent checks passed 8/8 on base and gold. Only the two gold-derived checks distinguished them.

Freeze an independent check that fails on base and passes on gold. Label gold-derived evidence. Verify inference, isolation, instruction loading and usage in a matched pilot before expanding.

WHY THE HISTORICAL RUN STOPPED

After repeated service limits, the run paused and its restart was canceled. The 20 attempted arms used 9,061.0 recorded seconds and at least 25,060,839 tokens. Usage is a lower bound for 5 arms. All 10 human references lack time and usage data.

The first continuation recovered 16 base/gold snapshots and the pinned Linux CLI. Four Tokio role profiles each passed 14 checks directly and in descendants. Device sign-in was unavailable, the pinned server had no authenticated account and desktop builds differed. At the user's request, that continuation stopped before a scored turn.